KrishiSaathi

Interview Defence Guide

Every resume line explained, evidenced and defensible —
with diagrams, plain-English terminology and an honest answer
to every awkward question. Rebuilt from the repository itself.

React · TypeScript · FastAPI · SQLAlchemy · Keras / TensorFlow · MobileNetV2

What changed from the first guide

- **Resume check.** Four claims on your KrishiSaathi bullets do not match the repository. Chapter 1 shows each one and the corrected wording.
- **Resume-first structure.** Chapter 4 takes your three resume bullets one by one: plain-English meaning, proof in the repo, terms, follow-ups.
- **Diagrams.** 13 figures: architecture, both request flows, transfer learning, data split, charts of the measured results, DAP netting, database, tests.
- **Glossary and 50+ questions.** Every technical term an interviewer might poke at, then a question bank with answers (plus the follow-ups under each resume bullet).

Prepared for **Snehil Rai** · generated 14 Oct 2026 from `KrishiSaathi-main.zip`, your resume and the earlier guide.

Contents

01 Read this first: your resume vs your repository . . . 3
02 The project in 30 seconds, 90 seconds and one page . . . 5
03 Architecture and request flows . . . 7
04 Your resume lines, one by one . . . 10
05 Crop-health model deep dive . . . 15
06 Fertilizer planner deep dive . . . 19
07 API, database and frontend . . . 22
08 Testing and reliability evidence . . . 25
09 Glossary: terminology in plain English . . . 27
10 Question bank: more than 50 questions with answers . . . 29
11 Trap questions and honest limitations . . . 34
12 One-page cheat sheet . . . 36
13 Appendix A: run it and demo it in two minutes . . . 37
14 Appendix B: where things are in the repository . . . 38

How to practise with this guide

- **Day 1:** read Chapter 1 and fix your resume. Read Chapters 2 and 3 aloud (pitches and diagrams).
- **Day 2:** read Chapter 4 (your resume lines) and the glossary. Cover the answers and say each definition out loud.
- **Day 3:** run the app once (Appendix A), then work through the question bank without looking. Check the repo whenever you hesitate.
- **Interview morning:** read the one-page cheat sheet (Chapter 12) and the honest-limitations statement.

> **IN PLAIN ENGLISH**
> A defence guide is not a script. Interviewers can tell memorised answers from understood ones. For each topic learn **the problem it solves, how data flows through it, where it lives in the code, the trade-off, and the limitation**. The boxes in this guide follow that pattern.

01 Read this first: your resume vs your repository

I compared every number on your KrishiSaathi resume bullets with the repository (the model card, the generated model report, the training metrics file, the test-suite and the code). **Four claims are wrong** and one wording needs care. An interviewer who opens your GitHub will see the mismatch within a minute, so fix this before anything else.

Resume says	Repository says (source)	Verdict	Use this instead
27.1% accuracy on 236 held-out field images	27.1% (`baseline_top1 = 0.2712`, n = 236) in model_card.json and model_report.md	✓ **matches**	Keep
field accuracy improved to **75.5%**	**55.5%** (`top1 = 0.5551`) in model_card.json, field_model_metrics.json and docs/model_report.md	✗ **wrong**	55.5%
correct-crop accuracy from 47.0% to **85.0%**	47.0% to **75.0%** (`crop_top1 = 0.75`)	✗ **wrong** (47.0 is right)	47.0% → 75.0%
ECE from 0.072 to **0.049**	0.072 to **0.059** (`ece_after = 0.0594`)	✗ **wrong**	0.072 → 0.059
at a **70%** threshold, **85%** accuracy on **58%** of field images	At 70%: 79.3% accuracy on 39.0% of photos. At 80%: 85.7% on 26.7%. App cut-off 60%: 75.4% on 48.3%. No row gives 85% on 58%.	✗ **wrong**	60% cut-off: answers 48%, right 75% of the time
2,059 field and 1,520 laboratory images	PlantDoc train 2,059 (fit) + 281 (validation); PlantVillage 1,520 (fit)	✓ **matches**	Keep
Validated calculations across 80 test combinations	2 P-sources × 2 K-sources × 4 split counts × 5 dose sets = 80 parameterised cases; whole suite = 166 tests	✓ **matches**	Keep (add “166 tests”)
Tech line: React, FastAPI, Keras, **MySQL**	MySQL works only if you set DATABASE_URL. Default and tested database is SQLite.	⚠ **check**	“SQLAlchemy (SQLite; MySQL-ready)” unless you really ran it on MySQL

WATCH OUT

Where did the wrong numbers come from? The README comparison table in the repository prints 75.5% and 85.0%, yet the same README's “Known limitations” says 55.5%, and the generated report and model card say 55.5%. So the README table is stale and your resume appears to have copied it. I could not find 0.049 or “85% on 58%” in any file, so I cannot tell where those came from; treat them as unsupported.

The README also ends with leftover Git merge-conflict markers (`=======` and `>>>>>>> 7090b8d…`) plus a duplicate two-line stub. Delete them.

FIX BEFORE YOUR INTERVIEW

1. Edit the resume with the corrected bullets below and re-export the PDF.
2. In README.md change the table cells 75.5% → **55.5%** and 85.0% → **75.0%**, and delete the merge-conflict lines at the bottom. Commit and push.
3. If you have already sent the old resume, say it first: *“There is a typo on my resume: the measured field accuracy is 55.5%, not 75.5%; the report in the repo has the exact figures.”* Owning a mistake early costs little; being caught costs a lot.

Corrected resume bullets (copy-paste ready)

SUGGESTED BULLETS

• Evaluated a MobileNetV2 leaf classifier on 236 held-out PlantDoc field photos (27.1% exact-class accuracy), then fine-tuned it on 2,059 field and 1,520 laboratory images, raising field accuracy to 55.5% and correct-crop accuracy from 47.0% to 75.0%.

• Calibrated prediction confidence with temperature scaling (expected calibration error 0.072 → 0.059) and added a 60% abstention threshold: the app answers 48% of field photos and is right 75% of the time when it answers.

• Built a React + FastAPI fertilizer planner that converts N-P_2O_5-K_2O doses into product quantities and split schedules (netting out DAP's nitrogen), converts acre / guntha / hectare, and cites reference doses; verified the arithmetic with 80 parameterised test combinations (166 tests overall).

What I verified, and what I could not

Checked by me	Not checked by me
Read every backend file, the training scripts, the model card and the report. Ran the backend test-suite: **165 passed, 1 skipped** (the skipped one needs TensorFlow and the weights). Ran the real fertilizer engine to produce the worked examples in Chapter 6. Counted tables, routes, tests and parameter combinations in the code.	Re-measuring the model's accuracy (needs TensorFlow plus the PlantDoc and PlantVillage datasets). I quote the figures stored in model_card.json. Running the React app, its 32 test blocks or the production build. Anything about how you deployed the project or how many people used it.

02 The project in 30 seconds, 90 seconds and one page

30-second pitch

SAY IT LIKE THIS

“KrishiSaathi is a web app with two tools for farmers. The first screens a leaf photo for crop disease using a MobileNetV2 model I fine-tuned on real field photos; it says so honestly when it is not confident instead of guessing. The second turns a nutrient dose from a soil report into fertilizer quantities and an application schedule, with unit conversion for acres and gunthas. It is a React front end, a FastAPI back end, SQLAlchemy storage, and I tested it with 166 automated tests.”

90-second pitch

SAY IT LIKE THIS

“I built KrishiSaathi around two separate farmer workflows. In **Crop Health** a user uploads a leaf photo. The backend validates the file, runs the classifier, and decides between four outcomes: a diagnosis, ‘low confidence’, ‘no leaf found’, or ‘model unavailable’. The interesting part was the model. My first model scored 93% on clean lab photos but only 27% on real field photos, so I fine-tuned it with PlantDoc field images plus lab images and measured it on 236 field photos it had never seen. Exact-class accuracy rose to 55.5% and correct-crop accuracy to 75%. I calibrated its confidence and set a 60% cut-off, so it answers about half the photos and is right about three times in four when it answers.”

“The **Fertilizer Planner** is deliberately not an AI model. The farmer supplies a dose, or picks a cited reference dose, and the engine does the arithmetic: product quantities from nutrient grades, DAP's nitrogen netted out of the urea, area conversion, and a split schedule. I verified it with 80 parameterised test combinations. The main limitation is that both tools are decision aids: the crop model is modest on field photos and the planner does not decide the right dose.”

The one-page picture

	Crop Health	Fertilizer Planner
Problem	Farmer cannot tell what is wrong with a leaf, and a wrong guess costs money or crop	Farmer has a nutrient dose (kg/ha) but needs bags and kilograms for a real plot in acres or gunthas
Technique	Image classification (MobileNetV2, 39 outputs) + calibration + abstention + Grad-CAM	Deterministic arithmetic on fertilizer grades; no machine learning
Input / output	JPG, PNG or WebP photo → status, disease, confidence, evidence level, warnings, heat-map	N, P_2O_5, K_2O doses, area, sources, splits → product kg, schedule, warnings, sources
Headline number	55.5% exact class and 75.0% right crop on 236 held-out field photos	80 parameterised test cases + 166 tests in total
Honest limit	Small test set (±6 points); 27 of 38 classes have field evidence	Never chooses the dose; fixed standard product grades
Where	`backend/crop_model.py`, `main.py`	`backend/fertilizer_engine.py`, `fertilizer_data.json`

IN PLAIN ENGLISH

Why the split into a “model” and an “engine” matters. The classifier *predicts* from pixels and can be wrong; the engine *calculates* from numbers and must be exactly right. That is why the engine has exact-arithmetic tests while the classifier has an evaluation report. If an interviewer asks “is it all AI?”, this is the sentence: *“No. One module is a learned model with measured uncertainty; the other is plain, tested arithmetic.”*

Why this project suits a software / data-engineering role

SAY IT LIKE THIS

“The model was only part of the work. The rest was engineering: validating untrusted input, designing a typed API, persisting structured results, degrading gracefully when TensorFlow or the database is unavailable, keeping train, validation and test data strictly separate, generating the evaluation report from code instead of typing numbers by hand, and writing tests. Those are the same concerns as in a data pipeline: data quality, contracts, reproducibility and reliability.”

DON'T SAY

- “It diagnoses crop diseases accurately.” → It is a **screening aid**; field accuracy is 55.5% exact class.
- “It recommends the right fertilizer.” → It **converts a dose you give it** into products.
- “It is production-scale / has users.” → The repository shows an application and tests, not users, uptime or traffic.
- “90% accurate.” → Only on a 380-photo clean lab sample (90.0%). Always name the test set.

03 Architecture and request flows

If you can redraw these three diagrams from memory you can answer most “walk me through it” questions. Learn them in order: system, crop flow, fertilizer flow.

KrishiSaathi — system architecture

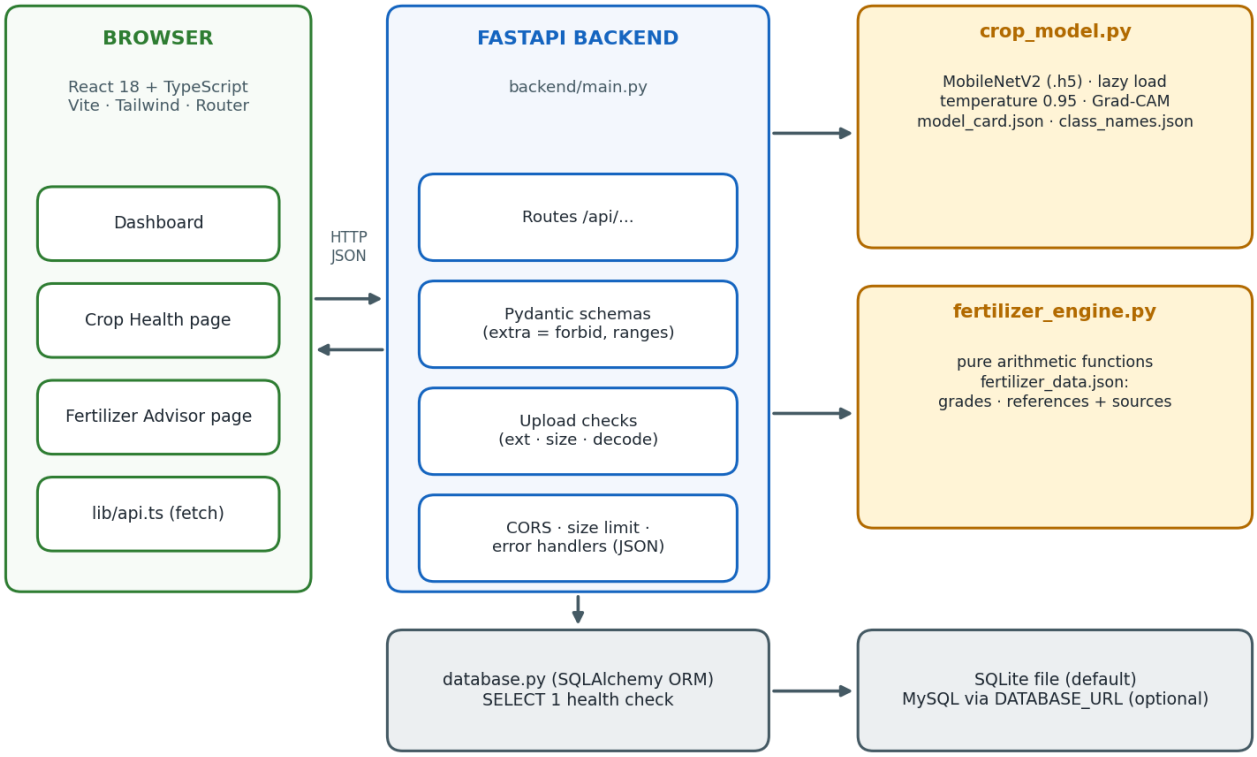
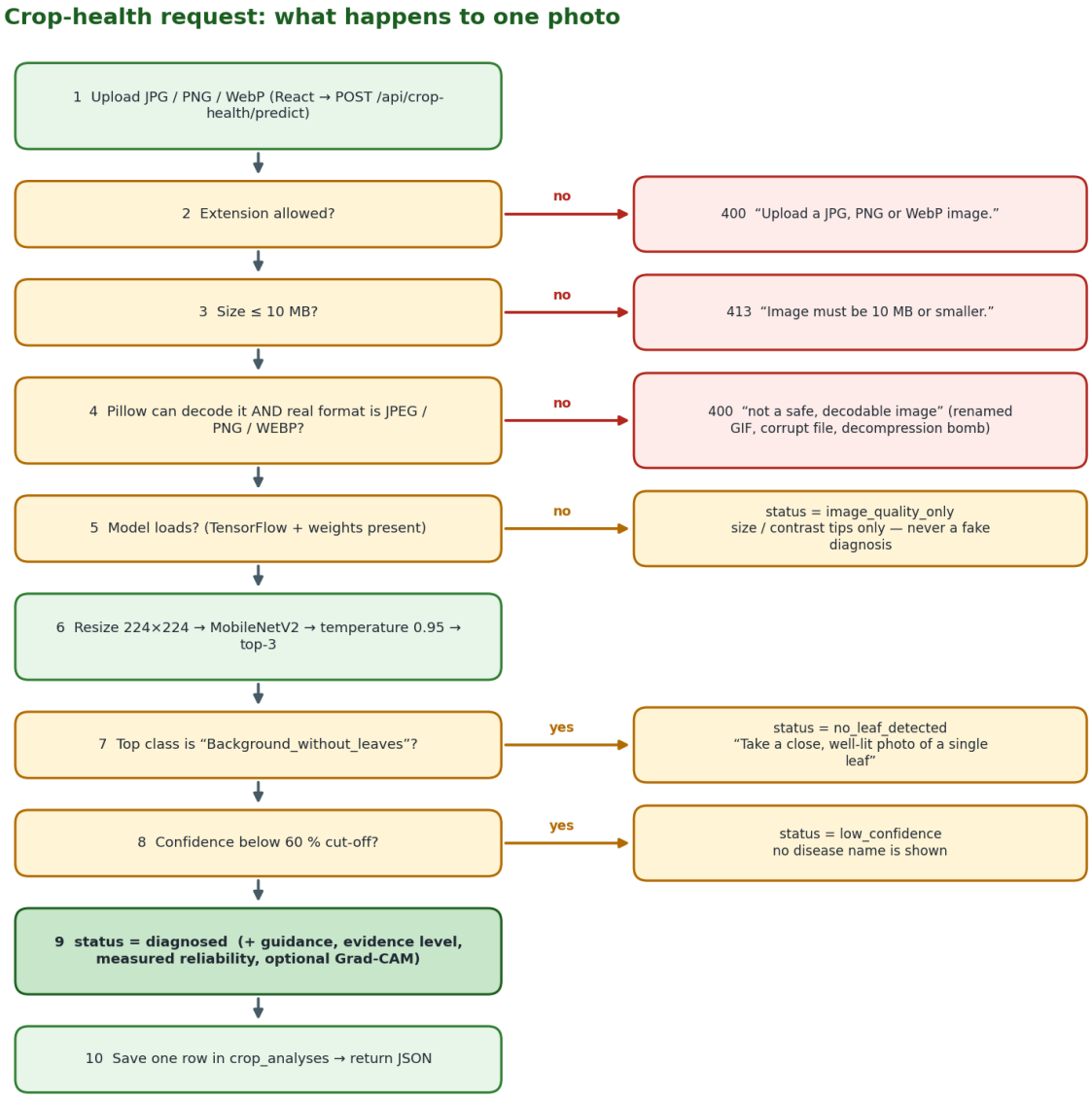


Figure 1. Four layers: browser UI, FastAPI API, the two engines (crop model and fertilizer engine), and the database. The engines never talk to the browser directly.

Layer	What it does	Key files
Frontend	Three pages (Home, Crop Health, Fertilizer planner): forms, loading and error states, result display, saved history. Validates input before sending; the server validates again.	`Frontend/src/pages/*.tsx`, `lib/api.ts`, `lib/fertilizer.ts`
API	Routes, upload checks, request validation (Pydantic), response shaping, error handling, CORS, body-size limit.	`backend/main.py`, `schemas.py`, `config.py`
Crop model	Loads MobileNetV2 lazily, preprocesses, predicts, applies temperature, decides confidence status, builds Grad-CAM.	`backend/crop_model.py`, `models/`
Fertilizer engine	Pure functions: unit conversion, product quantities, nitrogen netting, split schedule, soil ratings, warnings.	`backend/fertilizer_engine.py`, `fertilizer_data.json`
Persistence	SQLAlchemy models for crop analyses and fertilizer plans; SQLite by default, MySQL if DATABASE_URL says so; live health check.	`backend/database.py`
Evidence	Tests, training and evaluation scripts, generated model report.	`backend/tests/`, `training/`, `docs/model_report.md`

Crop-health request flow



Amber = decision · green = action · red = rejected request. Every accepted upload is saved, including the “no diagnosis” outcomes.

Figure 2. Ten steps from upload to saved record. Six exits are possible; only one of them is a diagnosis.

The four outcomes of a crop-health request

status	When	What the farmer sees
`diagnosed`	Model loaded, top class is a real crop condition, confidence ≥ 60%	Disease or “looks healthy” + score + evidence level (field or lab-only) + measured reliability for that score + screening-aid warning
`low_confidence`	Confidence below 60%	No disease name at all; advice to retake the photo or ask an agriculture officer
`no_leaf_detected`	Top class is the Background class	“No leaf was detected”; photo tips
`image_quality_only`	Model not available (TensorFlow or weights missing) or inference crashed	Size and contrast feedback only; the record is saved with null disease and confidence so it can never pass as a diagnosis

Fertilizer-plan request flow

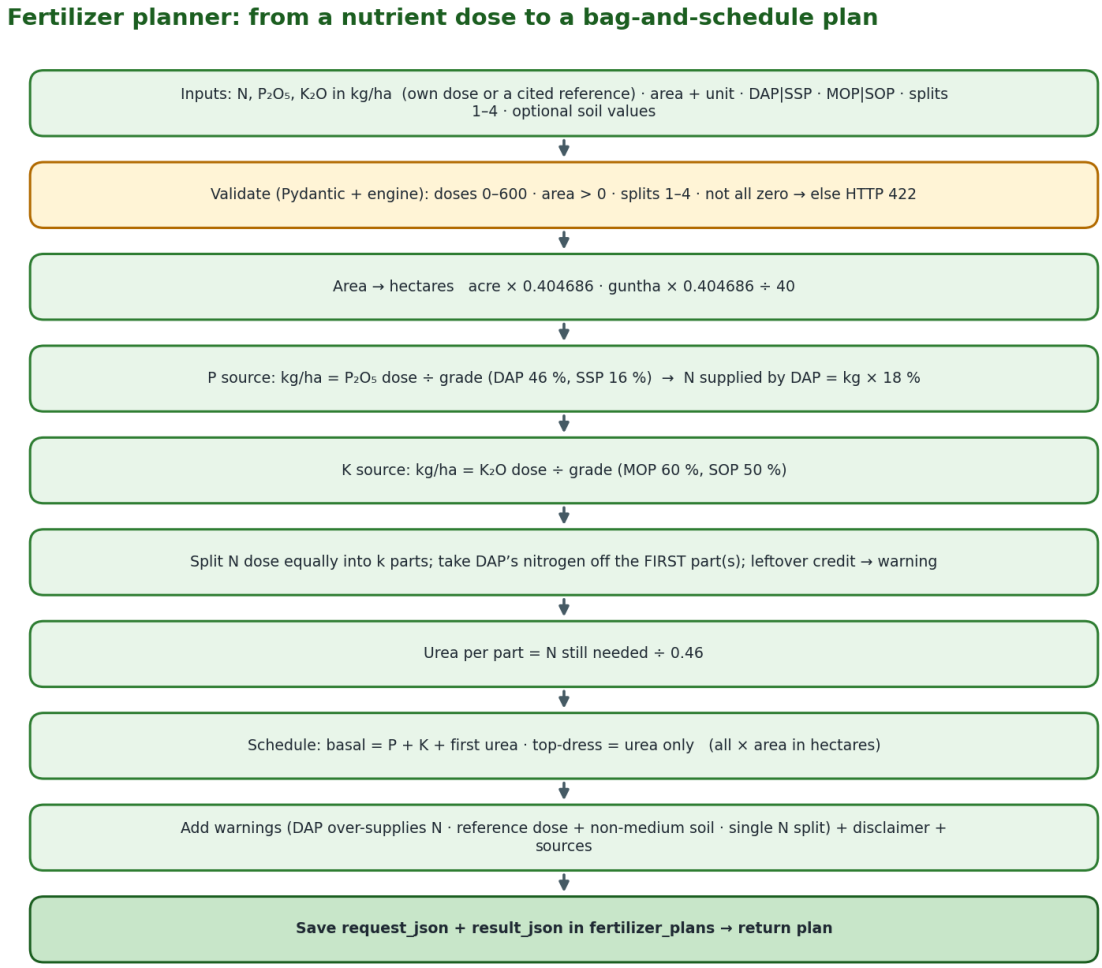


Figure 3. Every step is plain arithmetic on constants stored in fertilizer_data.json. Phosphorus is computed first because DAP also carries nitrogen.

Design principles you can claim (each is visible in the code)

- **Never fake an answer.** Missing model → image-quality feedback, not a guess. Low confidence → no disease name. Nullable database columns stop a non-diagnosis from looking like one.
- **One source of truth for constants.** Product grades, unit factors, rating limits and reference doses all live in `fertilizer_data.json` with sources, not scattered through code.
- **Validate at every boundary.** Browser form → Pydantic schema → engine checks. Typo guards (dose ≤ 600 kg/ha, area limits) are labelled as typo guards, not agronomic limits.
- **Degrade, don't crash.** TensorFlow is imported lazily; the API still starts without it; `/api/health` reports database and model status separately.
- **Show the uncertainty.** The UI displays the measured field accuracy for a score like yours and warns when this particular answer was usually wrong in testing.

04 Your resume lines, one by one

Each section below takes one bullet (in its **corrected** form from Chapter 1) and gives you: a phrase-by-phrase breakdown, proof in the repository, the terminology an interviewer may probe, and likely follow-ups with answers.

Bullet 1 · Evaluation and fine-tuning

THE LINE

Evaluated a MobileNetV2 leaf classifier on 236 held-out PlantDoc field photos (27.1% exact-class accuracy), then fine-tuned it on 2,059 field and 1,520 laboratory images, raising field accuracy to 55.5% and correct-crop accuracy from 47.0% to 75.0%.

Phrase	What it means	Proof
MobileNetV2 leaf classifier	A small convolutional neural network (CNN) that maps a 224×224 leaf photo to 39 scores: 38 crop-condition classes + one "Background_without_leaves" class, across 14 crops.	`crop_model.py`, `models/class_names.json`
236 held-out PlantDoc field photos	The official PlantDoc *test* split: real-world, web-scraped photos with clutter and uneven light. "Held-out" means the model never trained on them and they were never used to tune it.	`model_report.md` → Training data table
27.1% exact-class accuracy	The lab-only model named the exact crop *and* condition for 64 of 236 field photos.	`model_card.json` → baseline_top1
fine-tuned on 2,059 field + 1,520 lab	Continued training only the top layers on PlantDoc train photos (field) mixed with PlantVillage photos (lab) so it adapts to field conditions without forgetting lab conditions. Also used: 281 validation photos, 40 non-leaf photos, 150 synthetic backgrounds.	`train_field_model.py`
field accuracy 55.5%	131 of 236 photos exactly right. Margin of error about ±6 points because the test set is small.	`model_card.json` → top1
correct-crop 47.0% → 75.0%	Counts a photo as right if the *crop* is right even when the disease label is wrong: 111 → 177 of 236. Useful because crop identity is the first thing the app must get right.	`model_card.json` → crop_top1

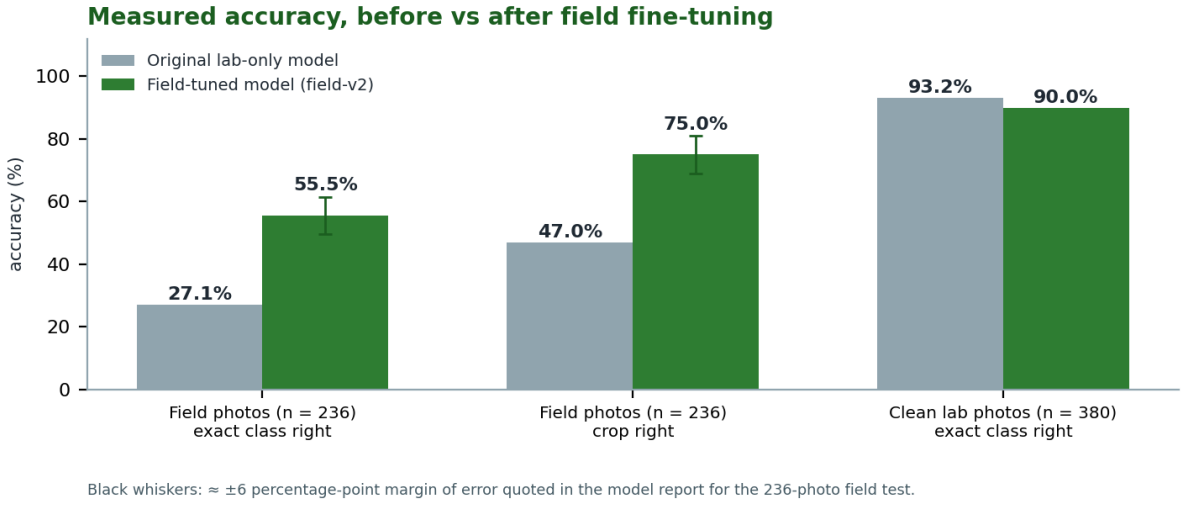


Figure 4. Fine-tuning more than doubled field accuracy; lab accuracy fell slightly (93.2% → 90.0%), which is a normal trade-off when a model is pulled towards a new domain.

IN PLAIN ENGLISH

Domain shift. A model learns the quirks of the photos it trains on. PlantVillage lab photos have one leaf, a plain background and even light. Real field photos have soil, hands, other leaves and shadows. A model that never saw those conditions is surprised by them, so its accuracy collapses (93% → 27%). Fine-tuning on field photos reduces that gap.

Held-out test set. Like an exam whose questions you never saw while studying. If you tune the model on the test photos, the score stops measuring anything.

What to know about the training data

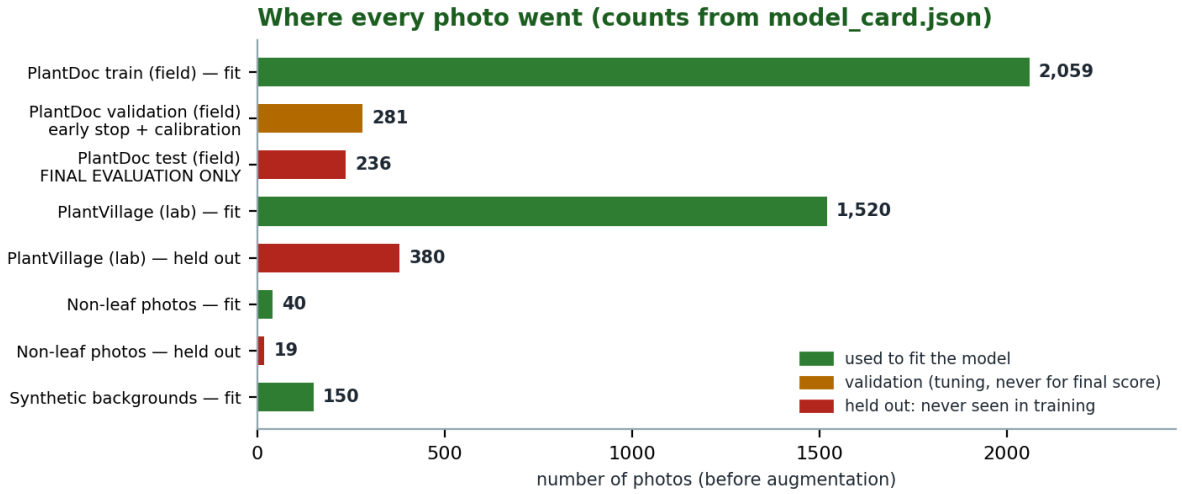


Figure 5. Every photo has exactly one role. Red bars are never seen by training or tuning.

Questions this bullet invites

Q. Why is 55.5% still low? Is that a good result?

"It is more than double the starting point, but it is still modest. The test set is only 236 photos, PlantDoc is web-scraped, I had about 2,000 field training photos, and some classes are very hard: tomato mosaic virus was recognised 0 of 10 times. I treat the model as a screening hint and the app says so. The important engineering point is that I measured honestly on photos the model never saw."

Q. Why not just train on all the data and report accuracy?

"Accuracy on training photos only tells you the model memorised them. A separate held-out test set is the only fair estimate of how it behaves on new photos. I also kept a validation split for early stopping and calibration, so the test set was touched exactly once, for the final evaluation."

Q. Did you start from ImageNet weights?

"The fine-tuning started from my existing lab-trained MobileNetV2 weights. Check your original training notebook before saying anything about ImageNet: the repository does not record how the first model was initialised, and I would not claim what I cannot show."

Q. What does 'fine-tuned' mean technically here?

"I froze the lower network up to the layer called block_12_add, cached its outputs for many augmented copies of each photo, and trained only the top blocks and the classifier on them, with batch-norm statistics frozen. That keeps training feasible on one CPU core and limits overfitting."

Bullet 2 · Calibration and the confidence threshold

THE LINE

Calibrated prediction confidence with temperature scaling (expected calibration error 0.072 → 0.059) and added a 60% abstention threshold: the app answers 48% of field photos and is right 75% of the time when it answers.

Phrase	What it means	Proof
Confidence	The top softmax score for the predicted class, between 0 and 1. It is *not automatically* a true probability.	`crop_model.predict`
Temperature scaling	Divide the model's raw scores (logits) by a single number T before softmax. T < 1 sharpens the probabilities, T > 1 softens them. T was fitted on the 281 validation photos and is 0.95.	`model_card.json` → temperature; `_calibrate()`
ECE 0.072 → 0.059	Expected calibration error: the average gap between stated confidence and actual accuracy, weighted by how many photos fall in each confidence bin. Lower is better. Measured on the 236 test photos.	`model_card.json` → ece_before/after
60% abstention threshold	If the top confidence is below 0.60 the app refuses to name a disease (status low_confidence).	`low_confidence_threshold` in the model card
answers 48%, right 75%	At 60%: 114 of 236 photos answered; 86 of those 114 were right (75.4%).	`selective` table in the model card

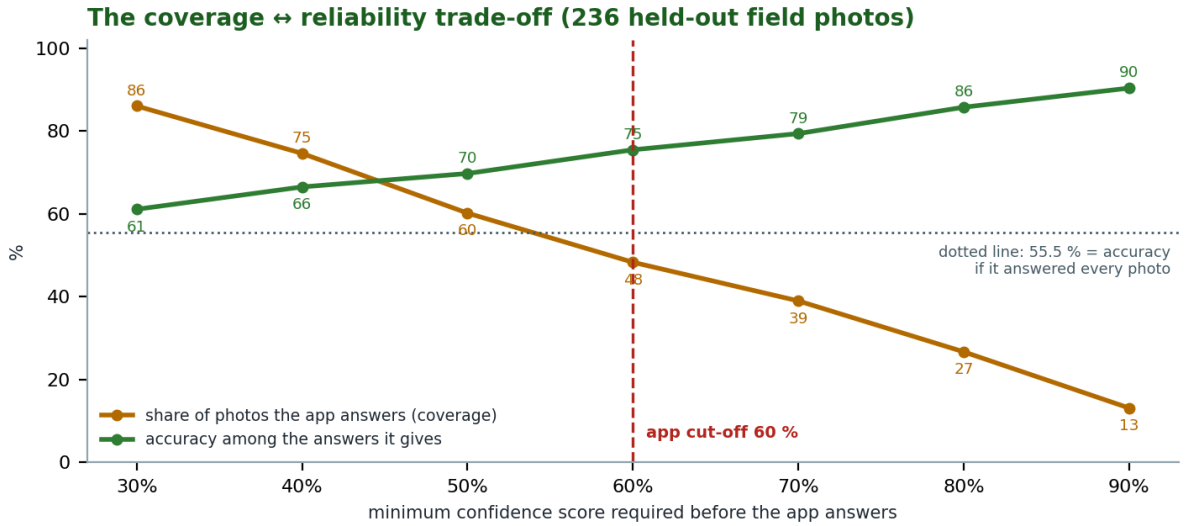
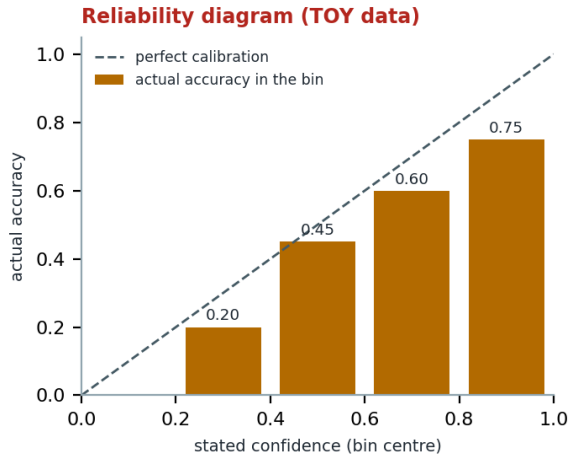


Figure 6. Raising the cut-off makes the answers more reliable but the app answers fewer photos. 60% is a policy choice, not a mathematical optimum.

Calibration in one picture (toy numbers)



ECE = Σ (share of photos × |confidence − accuracy|)

bin	share of photos	avg conf.	accuracy	gap	share×gap
0.2–0.4	0.10	0.30	0.20	0.10	0.010
0.4–0.6	0.20	0.50	0.45	0.05	0.010
0.6–0.8	0.40	0.70	0.60	0.10	0.040
0.8–1.0	0.30	0.90	0.75	0.15	0.045
				ECE =	**0.105**

Figure 7. Toy example, **not your data**. If a model says “80–100% sure” but is right only 75% of the time, there is a gap; ECE is the weighted average of those gaps.

Questions this bullet invites

Q. Why not just always show the top prediction?

“Because a confident-looking wrong answer can lead a farmer to spray the wrong thing. Abstaining below 60% trades coverage for reliability: on my test set the app answers about half the photos, but those answers were right 75% of the time instead of 55.5% overall.”

Q. How did you choose 60%?

"It is a policy choice, and the table shows the alternatives: at 80% it answers only 27% of photos but is right 86% of the time. I set 60% as a balance between being useful and being reliable. I would tune it with real user feedback and the cost of a wrong answer." (Say this honestly; do not claim an optimisation you did not run.)

Q. Does calibration make the model more accurate?

"No. Temperature scaling changes confidence, not which class wins, so accuracy is identical. It only makes the scores a more honest guide. ECE fell from 0.072 to 0.059, a small improvement, and I describe the confidence as a rough guide, not a probability."

Q. Your calibration set is small. Is that a problem?

"Yes. I fitted one temperature, a single parameter, on 281 validation photos, which limits overfitting, but the estimate is still noisy. That is why the app shows the measured accuracy for the score range instead of saying 'this is an X% probability'."

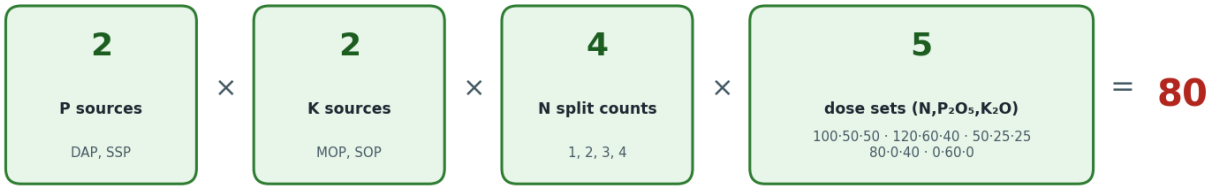
Bullet 3 · The fertilizer planner and its tests

THE LINE

Built a React + FastAPI fertilizer planner that converts N-P_2O_5-K_2O doses into product quantities and split schedules (netting out DAP's nitrogen), converts acre / guntha / hectare, and cites reference doses; verified the arithmetic with 80 parameterised test combinations (166 tests overall).

Phrase	What it means	Proof
N-P_2O_5-K_2O dose	The nutrient amounts, in kg per hectare, that a soil report or reference recommends. Phosphorus and potassium are quoted as oxide equivalents by convention.	`schemas.py` → PlanRequest
product quantities	Kilograms of urea, DAP or SSP, MOP or SOP needed, from each product's nutrient percentage (its grade).	`fertilizer_data.json` → products
split schedules	Phosphorus and potassium go on at sowing (basal); nitrogen is divided into 1–4 parts; the first part goes with the basal dose.	`build_plan()` → applications
netting out DAP's nitrogen	DAP is 18-46-0, so supplying the phosphorus dose also supplies nitrogen; that nitrogen is subtracted before calculating urea.	`test_dap_nitrogen_is_netted_out_of_urea`
acre / guntha / hectare	1 acre = 0.404686 ha; 1 guntha = 1/40 acre.	`to_hectares()`
cites reference doses	Four reference doses, each with a source title, URL and a verification level: three cotton doses (Maharashtra State package of practices) and irrigated wheat (MPKV Rahuri, marked *partly verified*).	`fertilizer_data.json` → references
80 parameterised combinations	One test function run 80 times: 2 P-sources × 2 K-sources × 4 split counts × 5 dose sets, checking products and schedule both deliver the requested nutrients.	`test_fertilizer.py`

Where "80 test combinations" comes from (test_products_deliver_exactly_the_requested_dose)



Each case checks that the products AND the application schedule deliver the requested P_2O_5, K_2O and N for a 2.5-acre plot (within rounding).

The whole suite is 166 tests (37 in test_api.py + 129 in test_fertilizer.py); on my run 165 passed and 1 was skipped.

Figure 8. The 80 comes from multiplying the four parameter lists; it is a systematic grid, not 80 hand-written tests.

Questions this bullet invites

Q. Is this machine learning?

“No, and deliberately. It is deterministic arithmetic, so it can be tested for exactness. Using ML here would add uncertainty where none is needed.”

Q. What does ‘parameterised test’ mean?

“One test body that pytest runs once for every combination in a list of parameters. Here the same assertion runs for all 80 combinations of phosphorus source, potassium source, number of splits and dose.”

Q. Does the planner tell the farmer how much to apply?

“It tells them how much *product* they need for a dose. The dose itself comes from the farmer's soil health card, lab report or a cited reference. The README and the app say clearly that it does not decide the dose.”

Q. You list MySQL on the resume. Did you use it?

“The code supports it through DATABASE_URL and the .env.example documents a MySQL URL, but the default is SQLite and the test-suite uses in-memory SQLite. I only say MySQL was used if I ran it on a MySQL server.” (If you never did, remove ‘MySQL’ from the tech line.)

05 Crop-health model deep dive

What the model is

Item	Fact (from the repository)
Architecture	MobileNetV2 (a lightweight convolutional network). Input: RGB image resized to 224×224, scaled to 0–1 (bilinear resize). Output: 39 softmax scores.
Classes	38 crop-condition labels from the PlantVillage naming scheme plus `Background_without_leaves`. 14 crops: Apple, Blueberry, Cherry, Corn, Grape, Orange, Peach, Bell pepper, Potato, Raspberry, Soybean, Squash, Strawberry, Tomato.
Label format	Labels look like `Tomato___Late_blight`; the code splits on ___ into crop and condition, and “healthy” as the condition means a healthy leaf.
Files	`models/crop_disease_field_v2.h5` (9.8 MB, used by the app) · `crop_disease_mobilenetv2_final.h5` (original lab-only model, fallback) · `model_card.json` (temperature, threshold, evidence levels, measured results).
Loading	Lazy: TensorFlow is imported on first use (and warmed in a background thread at start-up). If TensorFlow or the file is missing the API still runs.
Inference	One prediction at a time (a lock keeps memory use predictable); run in a thread-pool so the web server stays responsive.

IN PLAIN ENGLISH

CNN (convolutional neural network). Slides many small learnable filters across the image. Early layers pick up edges and colours, middle layers pick up textures such as spots or mould, later layers pick up shapes. A final layer turns those features into class scores.

Why MobileNetV2? It uses *depthwise separable convolutions* (a cheaper way of filtering) and *inverted residual blocks*, so it needs far less computation and memory than a large network. That suits a web service running on a CPU. I did **not** benchmark it against other architectures, so I do not claim it is the best choice, only a sensible, light one.

How the model was trained

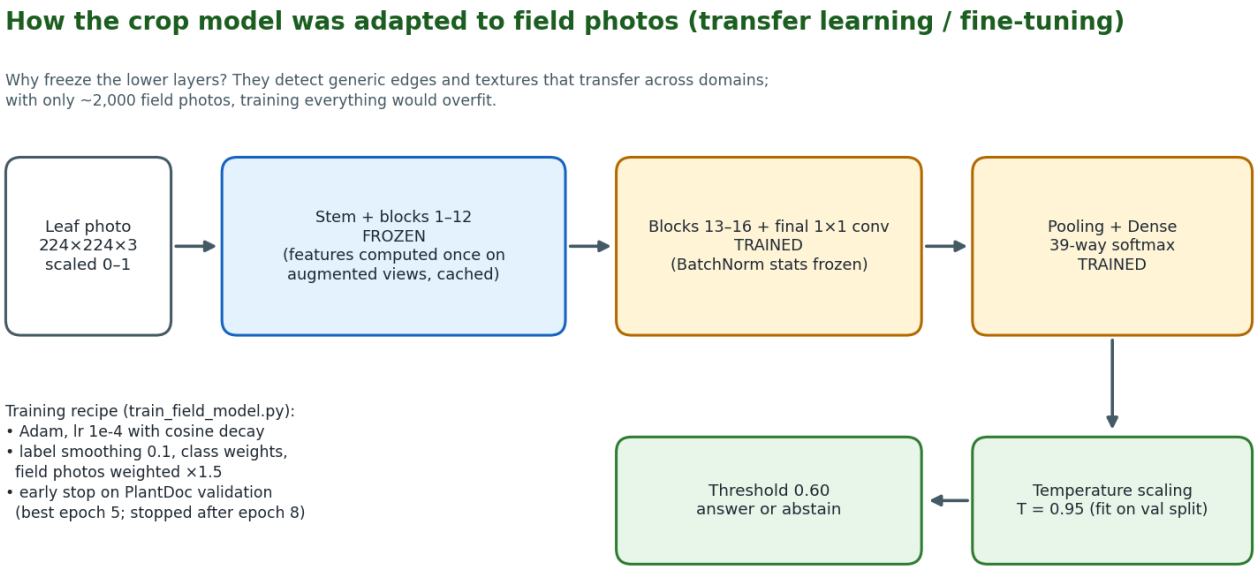


Figure 9. Lower layers are frozen and their outputs cached; only the top blocks and classifier learn from field photos.

Technique	Plain English	In your code
Transfer learning / fine-tuning	Start from a model that already knows something (here, the lab-trained model) and keep training on new data instead of starting from zero.	`train_field_model.py`

Technique	Plain English	In your code
Freezing layers	Stop some layers from changing. The early ones detect generic edges and textures, which are useful everywhere; freezing them stops them being ruined by a small dataset.	frozen up to `block_12_add`
Data augmentation	Make extra training copies of a photo with random crop, flip, rotation (±30°), brightness / contrast / colour changes and blur, imitating phone-camera variety.	`augmented_view()`: 3 views per field photo, 2 per lab photo (script defaults)
Label smoothing (0.1)	Train on 90% / 10% targets instead of 100% / 0% so the model does not become over-confident.	`CategoricalCrossentropy(label_smoothing=0.1)`
Class weights	Give rare classes more weight so common classes do not dominate: weight = √(largest class ÷ this class), clipped between 1 and 8. Field photos get an extra ×1.5.	`sample_weight`
Early stopping	Stop when validation accuracy stops improving (patience 3) and restore the best weights. Your run: best at epoch 5 (59.4% validation), stopped after epoch 8.	`field_model_metrics.json` → history
Batch-norm frozen	Batch-normalisation layers keep their stored statistics, which keeps training stable with small batches.	`layer.trainable = not BatchNormalization`
Synthetic backgrounds	150 generated images (flat colours, gradients, noise) plus 40 real non-leaf photos teach the Background class what “no leaf” looks like.	`synthetic_backgrounds()`
Adam + cosine decay	A standard optimiser with a learning rate of 1e-4 that shrinks smoothly during training.	`--lr 1e-4`

WATCH OUT

Fairness detail: the original model used nearest-neighbour resizing, the new one uses antialiased bilinear. The training README states each model was evaluated with its own preprocessing, so the comparison is like-for-like. Be ready to mention it if asked “is the comparison fair?”.

Reading the per-class results

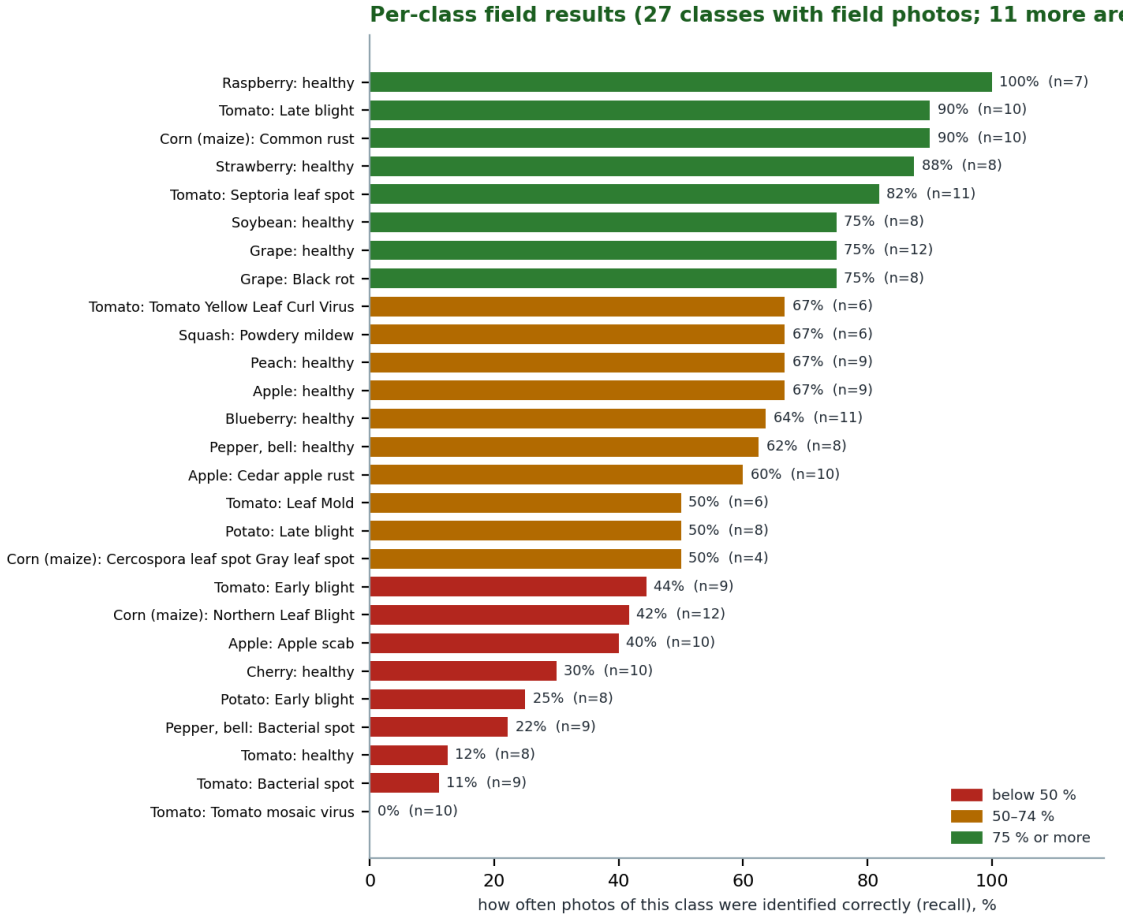


Figure 10. Recall per class on the 236 field photos. Several classes have under 12 test photos, so one photo moves the percentage by 8–12 points.

IN PLAIN ENGLISH

Recall (“Recognised” in the report): of the photos that truly are class X, how many did the model call X?

Precision (“Right when it said it”): of the photos the model called X, how many truly are X?

Real example from your report: tomato Septoria leaf spot has recall 81.8% (9 of 11 found) but precision only 40.9% (the model said “Septoria” 22 times and was right 9 times). So the model *over-uses* that label. Tomato healthy has recall 12.5% (1 of 8). Tomato mosaic virus: 0 of 10.

The app uses this: for 8 answers whose precision was under 50% (with at least 5 predictions), the response adds a warning such as “in field tests this answer was right only 20% of the time”. See `weak_answer_note()` in `main.py`.

Grad-CAM

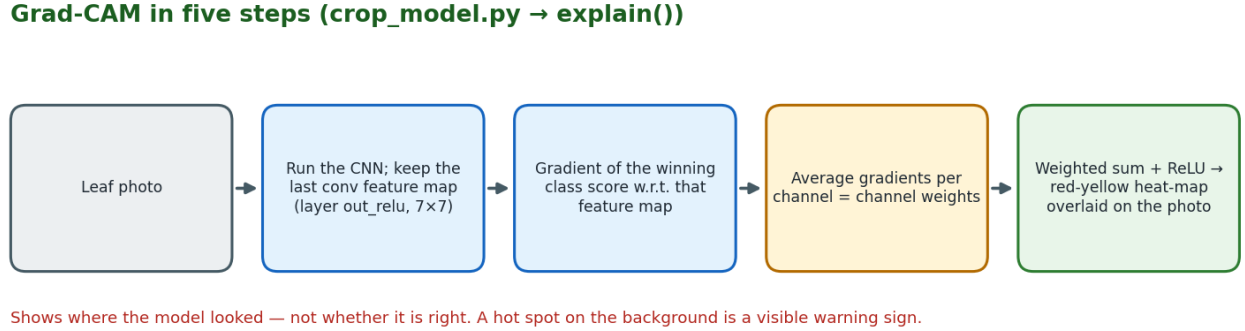


Figure 11. Grad-CAM highlights the image regions that pushed the model towards its answer.

Q. What does Grad-CAM prove?

“Nothing about correctness. It shows where the network looked. If the hot spot is on the leaf's lesions that is reassuring; if it is on the background, that is a visible warning. I show it as an inspection aid, not as an explanation or a guarantee.”

Evidence levels: which classes were actually tested on field photos

27 of the 38 disease classes have field photos in PlantDoc, so they were trained and tested on field conditions. The other **11 are lab-only** and the app labels them as such with a warning: Apple black rot; Cherry powdery mildew; Corn healthy; Grape esca (black measles); Grape leaf blight; Orange huanglongbing (citrus greening); Peach bacterial spot; Potato healthy; Strawberry leaf scorch; Tomato spider mites; Tomato target spot.

Non-leaf photos and unsupported crops

The background class plus synthetic backgrounds lets the model say “no leaf”. On the 19 held-out non-leaf photos none was shown as a confident result (0%), but 19 photos is tiny and mostly ordinary pictures and graphics: it proves little. Only 14 crops are supported; a photo of an unsupported plant, several leaves, soil or hands can still produce a confident wrong answer. The app says so in its guidance.

> **SAY IT LIKE THIS**
> “The strongest point of this model isn't the 55.5%. It is that I measured it on photos it never saw, found the original model failed in the field, reported the weak classes openly, and built the app to abstain and warn instead of overclaiming.”

06 Fertilizer planner deep dive

Agronomy vocabulary in one table

Term	Meaning	In the code
N, P, K	The three main plant nutrients: nitrogen, phosphorus, potassium. A farmer's soil report or crop guide gives a dose of each in kg per hectare.	`n, p2o5, k2o`
P_2O_5, K_2O	Phosphorus and potassium doses are quoted as oxide equivalents (a labelling convention), so "46% P_2O_5" on a bag means 46 kg of P_2O_5 per 100 kg of fertilizer.	`products[...]["P2O5"]`
Grade	The nutrient percentages of a product: urea 46-0-0, DAP 18-46-0, SSP 0-16-0, MOP 0-0-60, SOP 0-0-50.	`fertilizer_data.json`
Urea / DAP / SSP / MOP / SOP	Common nitrogen (urea), nitrogen + phosphorus (DAP), phosphorus (SSP, single super phosphate) and potassium (MOP muriate of potash, SOP sulphate of potash) fertilizers.	`products`
Basal / top-dress	Basal = applied at sowing or planting. Top-dressing = a later application on the growing crop.	`applications[].stage`
Soil Health Card	An Indian government soil-test report that rates nutrients low / medium / high and may recommend doses.	`soil_n, soil_p, soil_k`
Guntha	A land unit used in Maharashtra: 1/40 acre (about 101.2 m²).	`gunthas_per_acre = 40`

The core formula and a full worked example

For a product that contains a nutrient at fraction *f* (for example urea is 0.46 nitrogen): **product mass = nutrient needed ÷ f**. The engine applies this three times (phosphorus, potassium, nitrogen), with one twist: DAP also carries nitrogen, so phosphorus is done first and its nitrogen is subtracted before urea is calculated.

WORKED EXAMPLE

Example (computed with your real engine). Irrigated cotton reference dose 100 : 50 : 50 (N : P_2O_5 : K_2O kg/ha), 2.5 acres, DAP + MOP, 3 nitrogen splits.

Step	Calculation	Result
1 Area	2.5 acre × 0.404686	1.0117 ha
2 DAP for phosphorus	50 ÷ 0.46	108.7 kg/ha → supplies 108.7 × 0.18 = **19.6 kg N/ha**
3 MOP for potassium	50 ÷ 0.60	83.3 kg/ha
4 Nitrogen per split	100 ÷ 3 = 33.3 kg N each; DAP's 19.6 kg is taken off the **first** split	13.8 | 33.3 | 33.3 kg N/ha
5 Urea per split	each ÷ 0.46	29.9 | 72.5 | 72.5 kg/ha (total 174.9)
6 Scale to the plot	× 1.0117	**Basal:** DAP 110.0 + MOP 84.3 + urea 30.3 kg · **Top-dress 1:** urea 73.3 kg · **Top-dress 2:** urea 73.3 kg
Check	DAP 110.0 × 0.46 = 50.6 P_2O_5 ✓ · MOP 84.3 × 0.60 = 50.6 K_2O ✓ · N = 110.0 × 0.18 + 176.9 × 0.46 = 19.8 + 81.4 = 101.2 ✓	= 100 kg/ha × 1.0117 ha in every case

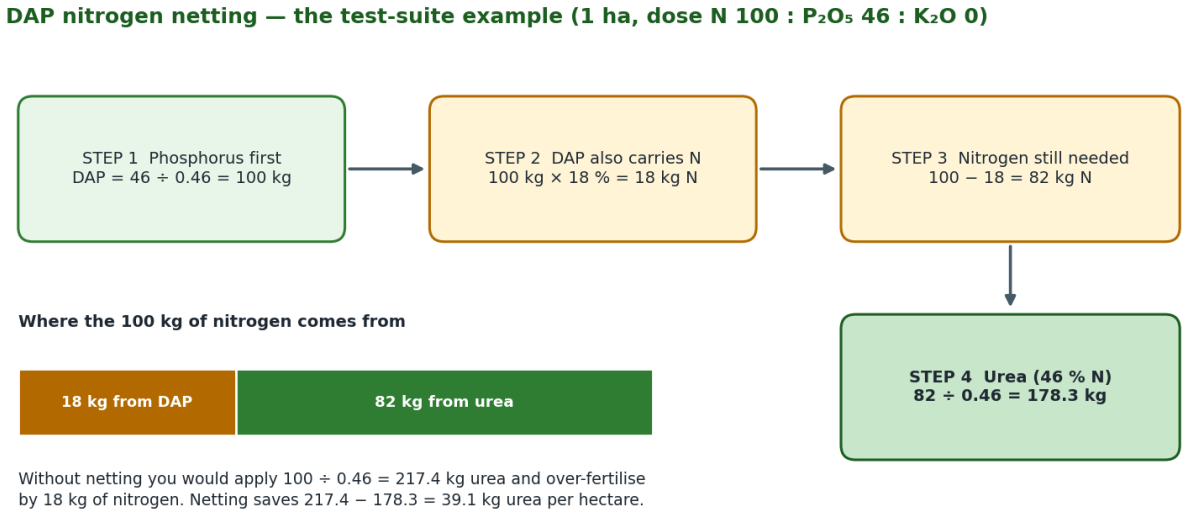


Figure 12. The 100/46 example asserted in the test-suite. Netting avoids over-applying 39 kg of urea per hectare.

WATCH OUT

Detail an interviewer may probe: the DAP nitrogen credit comes off the *first* nitrogen split because DAP is applied at sowing, not spread equally across all splits. That is why the basal urea in the example is only 30.3 kg while each top-dress is 73.3 kg. Nitrogen is split into **equal** parts; real crop guides often use unequal parts, and the app says split timing is generic.

Two more cases from the engine

Case	Input	Result
Guntha conversion	40 kg N/ha, 20 guntha, one split	20 guntha = 0.5 acre = 0.2023 ha → urea 87.0 kg/ha → **17.6 kg** for the plot
DAP over-supplies nitrogen	N 10, P_2O_5 60, DAP, 1 ha	DAP needed = 130.4 kg supplying 23.5 kg N (13.5 more than asked) → **warning**, no urea in the plan. Suggestion: use SSP or check the dose with an agriculture officer

Reference doses and how verified they are

Reference id	Crop (region)	N : P_2O_5 : K_2O	Verification	Source
`cotton-irrigated-mh`	Cotton, irrigated (Maharashtra)	100 : 50 : 50	verified	Maharashtra State Package of Practices for Cotton
`cotton-rainfed-hybrid-mh`	Cotton, rainfed hybrids	80 : 40 : 40	verified	same package
`cotton-desi-hirsutum-mh`	Cotton, desi and hirsutum varieties	50 : 25 : 25	verified	same package
`wheat-irrigated-mpkv`	Wheat, irrigated	120 : 60 : 40	**partly verified** (source text only partly readable)	MPKV Rahuri wheat recommendations 2019-20

The split counts shown with a reference (3 for irrigated cotton, 2 for the others) are defaults; only the cotton note about three splits comes from the source. Never describe all four as “fully verified”.

Soil ratings

If the farmer enters soil-test values, the engine rates each nutrient with default limits: nitrogen low below 280 and high above 560; phosphorus (Olsen) low below 10 and high above 25; potassium low below 120 and high above 280 (kg/ha). Boundary values count as **medium**. A rating printed on the farmer's own report **wins** over a computed one. Limits differ between states and labs (the data file notes TNAU's 240/480, 11/22, 110/280), so the app shows the caveat. When a reference dose is used and any soil rating is not medium, the plan warns the farmer to prefer the dose on their own card.

Validation and edge cases

Rule	Where	Result
Each dose between 0 and 600 kg/ha (a typo guard, not an agronomic limit)	`schemas.py` and `build_plan()`	422
At least one nutrient dose above zero; own-dose mode needs all three fields (use 0 for none)	Pydantic model validator	422
Splits must be 1–4; area > 0 and within 0.001–4000 ha	Field constraints + engine	422
NaN and Infinity rejected; unknown extra fields rejected (extra = forbid)	Pydantic	422
Reference mode needs a known reference_id	model_validator	422
Invalid requests are never saved to the database	`test_invalid_plan_requests_are_rejected_and_not_saved`	history stays empty

Honest limits of the planner

- It never derives a dose from soil values or crop type. It does not model organic sources, prices, soil type or season.
- Product grades are fixed standard values; the README tells farmers to check the grade on their bag, but the API has no field to change a grade.
- Nitrogen is split into equal parts; split timing is generic and the app tells the user to follow their crop's package of practice.
- Only four reference doses exist, and one is only partly verified. The README explains that other crops were not added because no source could be verified.

07 API, database and frontend

The API (all routes in `backend/main.py`)

Method and path	Purpose	Notes
`GET /api/health`	Database readiness (live `SELECT 1` + tables created) and model status	Returns 503 with details when the database is not ready
`GET /api/health/live`	Liveness only; does not touch the database	Used to tell “process is up” from “fully ready”
`POST /api/crop-health/predict`	Multipart field `image`; optional `?explain=true` adds a Grad-CAM data-URL	400 bad file · 413 too large · 422 missing field · 503 database down
`GET /api/crop-health/history`	Last 50 analyses with their closest matches	Newest first
`GET /api/crop-health/model`	Model status and the measured results from the model card	Lets the UI show real numbers instead of hard-coded ones
`GET /api/fertilizer/options`	Reference doses, product grades, rating limits, each with sources	Feeds the planner form
`POST /api/fertilizer/plan`	Validates, calculates and saves a plan	422 on any invalid input; invalid requests are not stored
`GET /api/fertilizer/history`	Saved plans, newest first (last 50)	Skips rows whose JSON is unreadable

FastAPI also serves interactive documentation at `/docs` (useful in a live demo).

HTTP status codes used

Code	Meaning here
`400`	Bad upload: wrong extension, or the bytes are not really a JPEG / PNG / WebP image, or the image is not safely decodable
`413`	Payload too large: image over 10 MB (also rejected early from the Content-Length header)
`422`	Valid JSON or form but invalid values: negative dose, unknown field, bad unit, missing file field
`500`	Unexpected error: the browser sees a generic message, the stack trace goes to the server log only
`503`	Database unavailable or not initialised

How uploads are hardened

- **Extension allow-list** (jpg, jpeg, png, webp), then **content check**: Pillow must decode the bytes and report a real JPEG, PNG or WEBP format, which catches a renamed GIF or a text file.
- **Size**: the server reads at most limit + 1 byte; a middleware also rejects an oversized `Content-Length` before reading the body. Default 10 MB, configurable with `MAX_UPLOAD_MB`.
- **Decompression bombs**: Pillow's decompression-bomb warning is turned into an error so a tiny file that expands to a gigantic image is refused.
- **Quality hints**: shorter side under 256 px or very low contrast adds advice to retake the photo.
- **What this is not**: a complete security audit. There is no authentication, no virus scanning and no rate limiting.

Pydantic and validation

> **IN PLAIN ENGLISH**
> **Pydantic** turns a Python class into a validator. Declare `n: float | None = Field(ge=0, le=600)` and any request outside that range is rejected before your code runs. Here it also forbids unexpected fields (`extra="forbid"`), rejects NaN and Infinity, and runs a cross-field rule: in *reference* mode a valid reference_id is required; in *own* mode all three doses are required. FastAPI uses the same classes to generate the API documentation.

Database

Database: two tables written by SQLAlchemy

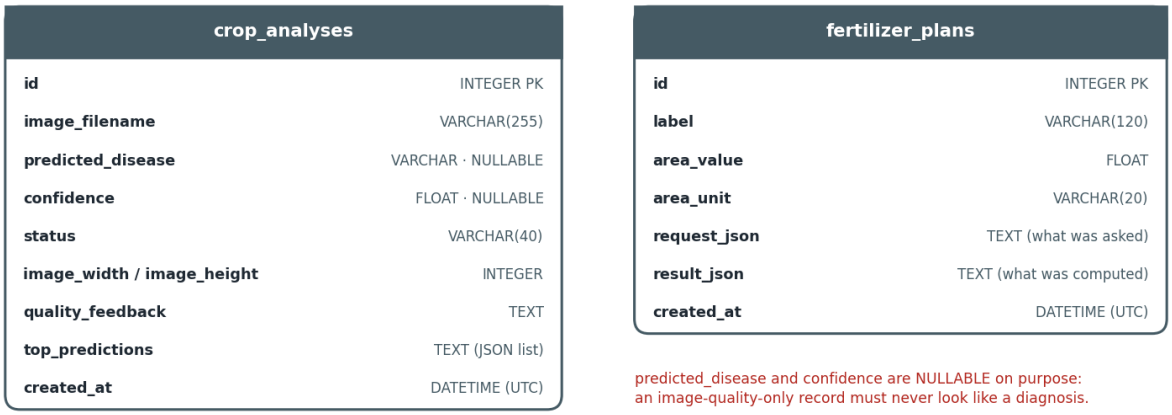


Figure 13. Two tables. The fertilizer table stores the request and the result as JSON text so a history entry can be shown exactly as it was calculated.

Topic	What the code does	Trade-off to mention
SQLite vs MySQL	Default is a local SQLite file; setting `DATABASE_URL` to a `mysql+pymysql://…` URL switches database with no code change (SQLAlchemy abstracts the dialect).	SQLite: zero setup, single file, fine for a portfolio demo, not for many concurrent writers. MySQL: server database; only claim it if you ran it.
ORM (SQLAlchemy)	Python classes `CropAnalysis` and `FertilizerPlan` map to tables; sessions are provided per request by `get_session()`.	Portability and safety (parameterised queries) vs a little overhead.
Schema changes	Start-up runs `create_all()` then an *additive* migration: `ALTER TABLE … ADD COLUMN` for columns introduced by earlier versions.	It is not a migration framework such as Alembic; there is no version history or rollback.
Health check	A live `SELECT 1` proves the database can be reached; a flag proves tables were created.	Configuration alone is not readiness.
JSON in a text column	request_json and result_json keep the exact inputs and outputs together.	Easy history and audit; not queryable column by column. If analytics were needed I would normalise the fields.
What is stored	Filename, image size, status, disease, confidence, top-3 matches, timestamps. **The image itself is not stored.**	Less storage and privacy exposure; you cannot re-run old photos.
Timestamps	Stored as naive UTC; the API appends “Z” so the browser converts to local time correctly.	Avoids the classic time-zone bug.

Reliability and error-handling patterns

Pattern	Where	Why
Lazy model loading + background warm-up	`crop_model.load()`, FastAPI lifespan	TensorFlow is slow to import and optional; the app should start without it
Graceful degradation	status `image_quality_only`	Better to say “no diagnosis available” than to guess or crash
Thread-pool inference + lock	`run_in_threadpool`, `RLock`	A blocking prediction must not freeze the web server; one at a time keeps memory predictable (but limits throughput; not load-tested)
Generic 500 handler	`unhandled_error()`	Log details server-side, never leak paths or stack traces to the browser
Validation-error handler	`validation_error()`	The default body echoes raw input (which could be NaN or unwelcome); this returns only location and message
CORS allow-list	`CORSMiddleware`, `CORS_ORIGINS`	Browser on port 8080 calling the API on port 8000 is a different origin and is blocked unless allowed; methods limited to GET and POST

WATCH OUT

README admits: **no authentication**; history is shared by everyone who can reach the API, so deploy only on a trusted network. Say this yourself before an interviewer finds it. Also restrict `CORS_ORIGINS` to your real front-end domain when deploying.

Frontend

Topic	What you can say
Stack	React 18 + TypeScript + Vite (dev server and bundler) + Tailwind CSS (utility classes) + React Router (client-side pages). Icons from lucide-react.
Structure	`App.tsx` (layout, navigation, routes) → pages `Dashboard`, `CropHealth`, `FertilizerAdvisor` → shared `components/ui.tsx`, `PlanView.tsx` → helpers in `lib/` (`api.ts`, `fertilizer.ts`, `crop.ts`).
State	Local component state with `useState` for file, preview, busy, progress, error, prediction, history. No global state library, which keeps a two-page app simple.
Upload	`CropHealth.tsx` sends the photo with an XMLHttpRequest so it can show upload progress, and calls `/api/crop-health/predict?explain=true` when the heat-map box is ticked.
Validation	`lib/fertilizer.ts` mirrors the server rules (dose ≤ 600, area > 0, soil value limit) for fast feedback; the server repeats every check because the browser can be bypassed.
Accessibility	Skip-to-content link, `aria-current` on navigation, `role="status"` for results, `aria-describedby` for upload tips, screen-reader-only labels.
Tests	Vitest + Testing Library: 32 test blocks across App, CropHealth and FertilizerAdvisor (I did not run them). Scripts: `npm run lint`, `typecheck`, `test`, `build`.
Config	`VITE_API_URL` selects the backend URL (default `http://127.0.0.1:8000`).

08 Testing and reliability evidence

What the 166 backend tests actually are

File	Tests	What they check
`tests/test_api.py`	37	Health and liveness; invalid extension, bytes and oversize uploads; model-unavailable fallback; confident, healthy, low-confidence and no-leaf predictions; history; database-down and unconfigured states; WebP accepted; renamed-file rejection; Grad-CAM only when requested; lab-only warning; reliability and weak-answer warnings; generic 500s
`tests/test_fertilizer.py`	129	**80** grid cases of exact delivery (products and schedule); **12** rating-boundary cases (e.g. 279.9 low, 280 medium, 560 medium, 560.1 high); **7** engine rejections; **16** invalid API requests that must not be saved; **14** single tests (DAP netting, SSP, overshoot warning, basal vs split, unit conversion, references have sources, NaN / Infinity, history, soil ratings)

Result of my run: 165 passed, 1 skipped in under a second. The skipped test `test_real_model_loads_and_returns_valid_scores` loads the real network and is skipped when TensorFlow or the weights are missing (as in my sandbox). A green suite does not prove that optional test ran on your machine; run it yourself before claiming the model integration is tested.

Testing techniques you can name

Technique	Plain English	Example in the project
Unit test	Test one small function with known input and output	`test_area_units`: 40 guntha equals 1 acre
Invariant / property check	Instead of one hard-coded answer, assert something that must always be true	Nutrients delivered by the products ≈ nutrients requested, for all 80 combinations
Boundary-value test	Test just below, on and just above each limit	Soil rating at 279.9, 280, 560, 560.1
Negative test	Feed invalid input and check it is refused cleanly	16 bad plan requests all return 422 and save nothing
Integration / API test	Call the real HTTP endpoints through a test client	FastAPI `TestClient` posting plans and uploading images
Fixture	Reusable test setup that is created and torn down per test	`client` gives each test its own in-memory SQLite database
Fake / stub / mock	A stand-in for something slow or unavailable	`FakeCropModel` replaces TensorFlow so tests are fast and deterministic
Monkeypatch	Temporarily replace a value or function inside a test	Change `NPK_MAX_VALUE` to 100 and check the new limit applies
Parameterisation	Run one test body over a list of inputs	`@pytest.mark.parametrize` ×4 stacked = 80 cases

IN PLAIN ENGLISH

Why a fake model? The API tests check the *application's decisions* (which status for which confidence, what gets saved, what is warned) with predictions you control. They deliberately do not test the neural network, whose quality is covered by the evaluation report, not by pytest.

What the tests do NOT prove

- That the model performs well on every class (the report shows it does not) or that the real model loaded on your machine.
- That the app is secure against attacks, handles heavy traffic, or has been deployed anywhere.
- That the front end works end to end in a browser: the front-end tests run in a simulated DOM, and I did not run them.

- That fertilizer doses or reference values are agronomically right. The tests prove the *arithmetic* (products deliver the requested nutrients), not the advice.

Continuous integration

A GitHub Actions workflow (`.github/workflows/ci.yml`) runs on every push to main and every pull request: backend job (Python 3.12, install requirements, `python -m pytest -q`) and frontend job (Node 20, `npm ci`, lint, typecheck, test, build). I have not seen it run, so check the Actions tab is green before you cite it.

Run the tests yourself

```
cd backend
python -m venv .venv && .venv\Scripts\activate   (Windows)
pip install -r requirements.txt
python -m pytest -q

cd ../Frontend
npm install
npm run lint && npm run typecheck && npm test && npm run build
```

09 Glossary: terminology in plain English

Each row gives a definition you can say out loud in one breath and, where possible, the place in your project where the idea appears. Alphabetical.

Term	Plain-English meaning	Where / how to link it to your project
Abstention (selective prediction)	Declining to answer when unsure, trading coverage for reliability.	Below 60% confidence the app shows no disease name.
Accuracy (top-1)	Share of photos where the single highest-scoring class is exactly right.	55.5% (131 of 236) on field photos.
Augmentation	Creating varied training copies of a photo (crop, flip, rotate, brightness, blur).	`augmented_view()`
Background class	An extra output meaning “no leaf here”.	`Background_without_leaves`, index 4 of 39.
Basal / top-dress	Fertilizer applied at sowing / on the growing crop later.	Planner schedule stages.
Batch normalisation	A layer that rescales its inputs to keep training stable; it keeps running statistics.	Frozen during fine-tuning.
Calibration	Making stated confidence match real accuracy.	Temperature 0.95; ECE 0.072 → 0.059.
CNN	Neural network that detects patterns with sliding filters; the standard tool for images.	MobileNetV2 is a CNN.
Confidence	The top softmax score; a rough guide, not a guaranteed probability.	Shown to the user with measured reliability.
CORS	Browser rule that blocks a page from calling another origin unless the server allows it.	`CORS_ORIGINS`: port 8080 → 8000.
Coverage	Share of inputs the system answers rather than abstains on.	48.3% at the 60% cut-off.
Correct-crop accuracy	Crop identity right even if the disease label is wrong.	47.0% → 75.0%.
DAP	Di-ammonium phosphate, a 18-46-0 fertilizer (N-P_2O_5-K_2O).	Its 18% nitrogen is netted out of urea.
Decompression bomb	A small file that expands into an enormous image and exhausts memory.	Pillow warning turned into an error.
Domain shift	Test conditions differ from training conditions, so accuracy drops.	Lab 93% → field 27%.
ECE	Expected calibration error: weighted average gap between confidence and accuracy per bin.	Lower is better; 0.059 here.
Early stopping	Stop training when validation stops improving; keep the best weights.	Best at epoch 5; stopped after 8.
Endpoint / REST API	A URL plus HTTP method that performs one operation and returns JSON.	`POST /api/fertilizer/plan`
Evidence level	Whether a class was trained and tested on field photos or lab photos only.	27 field, 11 lab-only.
Fine-tuning	Continuing to train an already-trained model on new data.	Top blocks trained on field + lab photos.
Fixture / mock / fake	Reusable test setup / stand-in object.	`client` and `FakeCropModel`.
Freezing	Preventing chosen layers from being updated in training.	Layers up to `block_12_add`.
Grade (fertilizer)	Nutrient percentages of a product.	Urea 46-0-0, DAP 18-46-0.
Grad-CAM	Heat-map of image regions that most influenced a class score.	`explain()` in `crop_model.py`

Term	Plain-English meaning	Where / how to link it to your project
Held-out test set	Data never used for fitting or tuning; gives an honest score.	236 PlantDoc test photos.
Hectare / acre / guntha	Land units: 1 ha = 2.471 acre; 1 acre = 0.404686 ha; 1 guntha = 1/40 acre.	`to_hectares()`
Inference	Using a trained model to predict on new input.	`CropModel.predict()`
Label smoothing	Training on slightly softened targets to avoid over-confidence.	0.1.
Lazy loading	Loading something only when first needed.	TensorFlow and the model weights.
Logits	Raw model scores before softmax.	Divided by T in temperature scaling.
MobileNetV2	A small, efficient CNN using depthwise separable convolutions and inverted residuals.	The backbone, 224×224 input.
N, P, K	Nitrogen, phosphorus, potassium: the main plant nutrients.	Planner inputs.
ORM / SQLAlchemy	Maps Python classes to database tables so you write Python, not raw SQL.	`database.py`
Overfitting	Memorising the training data and failing on new data.	Guarded by freezing, augmentation, early stopping.
Parameterised test	One test run over many inputs.	80-case grid.
PlantVillage / PlantDoc	Public leaf datasets: PlantVillage is clean lab photos; PlantDoc is web-scraped real-world photos.	Lab training / field training and test.
Precision / recall	Precision: of those called X, how many are X. Recall: of all true X, how many were found.	Tomato Septoria: recall 81.8%, precision 40.9%.
Pydantic	Library that validates data against typed models.	`schemas.py`
Softmax	Turns raw scores into positive numbers that sum to 1.	Final layer, 39 outputs.
SQLite / MySQL	File-based database with no server / full database server.	Default / optional via DATABASE_URL.
Temperature scaling	Divide logits by one number T before softmax to fix over- or under-confidence.	T = 0.95.
Transfer learning	Reusing what a model learned on one task for another.	Lab-trained → field-tuned.
Train / validation / test	Fit on train, tune and early-stop on validation, report once on test.	2,059 / 281 / 236 PlantDoc photos.
Typo guard	A limit that catches obvious mistakes (an extra zero), not a scientific rule.	Dose ≤ 600 kg/ha.
Vite / Tailwind / TypeScript	Fast front-end build tool / utility-class CSS / JavaScript with types.	Frontend stack.
Urea, SSP, MOP, SOP	Nitrogen (46%) / phosphorus (16% P_2O_5) / potash (60% K_2O) / sulphate of potash (50% K_2O).	Planner products.

10 Question bank: more than 50 questions with answers

Chapter 4 already covers the questions tied to your three resume bullets. These cover everything else. Answers are deliberately short: practise saying them in 20–40 seconds, then be ready to go deeper.

A. About the project

Q. 1. Walk me through the project end to end.

"A farmer opens the React app and picks Crop Health or the Fertilizer Planner. A leaf photo goes to FastAPI, which validates it, runs the MobileNetV2 model, applies the 60% confidence rule and saves the record. A plan request is validated by Pydantic, calculated by a pure Python engine and saved. Both results come back as JSON with warnings, and the UI shows them and keeps a history." (Redraw Figures 1–3.)

Q. 2. Why did you choose this problem?

Give your **real** reason in three beats: where the problem came from (family, region, a news story, a course), why it was worth solving (leaf diseases and fertilizer arithmetic are practical, high-stakes tasks), and what you learned (the gap between lab and field data). Do not invent a story.

Q. 3. Who is the user and how do they use it?

"A farmer or extension worker. They photograph a leaf on a phone and upload it, or enter the dose from a Soil Health Card with their plot size in acres or gunthas to get kilograms and a schedule. Plans can be printed and are saved in a history."

Q. 4. What was the hardest problem?

Pick one you can defend: **(a)** discovering the model dropped from 93% on lab photos to 27% on field photos and rebuilding the evaluation so it was honest; or **(b)** getting the DAP-nitrogen netting right and proving it with invariants.

Q. 5. What are you most proud of?

"That the app is honest about uncertainty: it abstains, shows measured reliability, labels lab-only classes and says what it cannot do."

Q. 6. How much did you write yourself? Did you use AI tools?

Answer **truthfully and specifically**. Say what you designed and wrote, what came from libraries, datasets or assistants, and what you checked. Interviewers mainly want to know you understand it. A strong test: can you explain `CropModel.predict()` and `build_plan()` line by line, change a threshold, and add a test, with no notes? Practise exactly that.

B. Machine learning

Q. 7. What is MobileNetV2 and why that model?

"A lightweight CNN built from depthwise separable convolutions and inverted residual blocks, so it is fast and small (my weights file is 9.8 MB). It suits CPU-based inference. I did not benchmark other architectures, so I do not claim it is the best."

Q. 8. Transfer learning, fine-tuning, training from scratch: what is the difference?

"From scratch starts with random weights and needs lots of data. Transfer learning reuses a model trained elsewhere. Fine-tuning continues training some of its layers on your data. I fine-tuned the top blocks of my lab-trained model on field plus lab photos."

Q. 9. What is domain shift?

"Test data looks different from training data. My lab-trained model scored 93.2% on clean lab photos but 27.1% on real field photos with clutter and uneven light."

Q. 10. How did you split the data?

"PlantDoc train photos were split into 2,059 for fitting and 281 for validation (early stopping and calibration). The PlantDoc test split of 236 was used only for the final evaluation. PlantVillage contributed 1,520 photos to fit and a different 380 for a lab check. Non-leaf photos: 40 to fit, 19 held out."

Q. 11. Top-1 accuracy versus correct-crop accuracy?

"Top-1 needs the exact crop and condition. Correct-crop only needs the crop. On field photos: 55.5% versus 75.0%."

Q. 12. Is a softmax score a probability?

"It sums to 1 and is called a probability, but it is only trustworthy if the model is calibrated. Neural networks are often over- or under-confident, which is why I calibrated and describe the score as a rough guide."

Q. 13. Explain ECE.

"Group predictions by confidence. In each group compare the average confidence to the actual accuracy and take the gap. ECE is the average gap weighted by group size. Mine went from 0.072 to 0.059." (Figure 7 has a toy example.)

Q. 14. What does temperature scaling do? Why 0.95?

"It divides the logits by one number T before softmax. T < 1 sharpens and T > 1 softens the scores. Fitting T on the validation photos gave 0.95, a small sharpening, and it reduced ECE. It does not change which class wins."

Q. 15. Precision versus recall using your results.

"Tomato Septoria leaf spot: recall 81.8% (found 9 of 11) but precision 40.9% (the model said Septoria 22 times, right 9 times). Recall asks 'did I find them?', precision asks 'when I said it, was I right?'."

Q. 16. How did you deal with class imbalance?

"Class weights equal to the square root of largest-class size over this class size, clipped between 1 and 8, plus an extra 1.5× weight on field photos. I also reported per-class results rather than only overall accuracy."

Q. 17. What is overfitting and how did you limit it?

"When the model memorises training photos. I froze the lower layers, used augmentation and label smoothing, trained on a small number of top layers, and stopped early on a separate validation split. Validation accuracy plateaued at about 59% from epoch 5."

Q. 18. How does it handle photos that are not leaves?

"There is a background class trained on 40 real non-leaf photos and 150 synthetic backgrounds. On 19 held-out non-leaf photos none was shown as a confident result, but that is a tiny test, and unsupported plants can still fool it."

Q. 19. What is the margin of error on 55.5%?

"About ±6 points: $1.96 \times \sqrt{(0.555 \times 0.445 \div 236)} \approx 6.3$ points. So the true field accuracy plausibly lies between roughly 49% and 62%."

Q. 20. Is 27.1% → 55.5% a real improvement or noise?

"The 28-point gap is much bigger than the ±6-point noise on 236 photos, so it is real. I did not run a formal paired significance test, and a few-point difference between two *fine-tuned* variants would be noise."

Q. 21. Is the 75.4% at the 60% cut-off solid?

"It is based on 114 answered photos (86 right), so its own margin of error is about ±8 points. I quote it as 'about three in four', not as a guarantee."

Q. 22. How would you improve the model?

"More field photos, especially from Indian farms and crops the model lacks; more examples for weak classes such as tomato mosaic virus and bacterial spot; better out-of-distribution detection; a larger test set; and checking results with agronomists."

C. Fertilizer planner

Q. 23. What are N, P and K and why P_2O_5 and K_2O?

"Nitrogen, phosphorus and potassium are the main plant nutrients. Fertilizer labels express phosphorus and potassium as oxide equivalents, so '46% P_2O_5' is the convention, not elemental phosphorus."

Q. 24. Walk me through the DAP example.

"For 100 N and 46 P_2O_5 per hectare: DAP = 46 ÷ 0.46 = 100 kg, which carries 18 kg of nitrogen. The remaining nitrogen is 82 kg, so urea = 82 ÷ 0.46 = 178.3 kg. Without netting I would apply 217.4 kg urea." (Figure 12.)

Q. 25. Why is phosphorus calculated first?

"Because DAP is both the phosphorus and a nitrogen source; you must know how much DAP is used before you know how much nitrogen is left for urea."

Q. 26. How are nitrogen splits handled?

"Nitrogen is divided into 1–4 equal parts. Phosphorus, potassium and the first nitrogen part go on at sowing. DAP's nitrogen is credited against the first part because it is applied then. Equal splits and generic timing are simplifications, and the app says so."

Q. 27. What if DAP supplies more nitrogen than the dose?

"The engine does not hide it. It omits urea and adds a warning that DAP supplies X kg more nitrogen than requested, suggesting SSP or checking the dose."

Q. 28. Where do the reference doses come from?

"Three cotton doses from the Maharashtra State Package of Practices and one wheat dose from MPKV Rahuri; each stores a source title, URL and verification level. The wheat one is only partly verified and the data file says so."

Q. 29. How do you convert gunthas?

"A guntha is 1/40 acre and an acre is 0.404686 hectares, so hectares = gunthas × 0.404686 ÷ 40. Everything is converted to hectares internally, then scaled back to the farmer's plot."

D. Backend and database

Q. 30. Why FastAPI? Your resume also lists Flask.

"FastAPI gives typed request validation through Pydantic, automatic API documentation and easy async support, which suited a project with strict input rules. Flask would also work but needs extra libraries for validation and docs." (Only discuss Flask in depth if you used it in another project.)

Q. 31. How is input validated?

"Three layers: the browser form, the Pydantic schema (ranges, allowed units, no extra fields, no NaN) and the engine's own checks. Invalid requests return 422 and are never saved."

Q. 32. How do you handle file uploads safely?

"Extension allow-list, a size limit enforced both from the Content-Length header and while reading, Pillow decoding to confirm it really is a JPEG, PNG or WebP, and decompression-bomb protection. It reduces risk but is not a complete security audit."

Q. 33. What if TensorFlow or the model is missing?

"The API still starts. The prediction route returns status image_quality_only with size and contrast advice, and records a null diagnosis. The health endpoint reports the model error."

Q. 34. How do you handle concurrency?

"Inference runs in a thread-pool so the event loop is not blocked, guarded by a lock so one prediction runs at a time. That keeps memory predictable but caps throughput; I have not load-tested it, and at scale I would put the model behind its own worker service."

Q. 35. Why SQLAlchemy and why SQLite by default?

"The ORM keeps database code portable: the same models work on SQLite and MySQL by changing DATABASE_URL. SQLite means no setup for a demo. For multiple users I would use a server database."

Q. 36. Why store request_json and result_json?

"So a history entry shows exactly what was asked and calculated even if the engine changes later. The trade-off is that fields are not individually queryable; for analytics I would normalise them."

Q. 37. How would you handle schema migrations?

"Right now there is a small additive migration that adds missing columns. For production I would adopt Alembic for versioned, reversible migrations."

Q. 38. How would you deploy and scale it?

"Containerise the API, serve the built front end as static files, move the model into a separate inference service, use a managed Postgres or MySQL, store images in object storage if needed, add authentication, rate limiting, restricted CORS and monitoring. Nothing in the repository shows I have deployed it, so I describe this as a plan."

E. Frontend and engineering practice

Q. 39. How does the frontend call the backend?

"Through fetch calls (and XMLHttpRequest for the upload, to get progress) to the base URL in VITE_API_URL. Because the page and API run on different ports, the backend's CORS allow-list must include the page's origin."

Q. 40. Why validate in both browser and server?

"Browser validation gives instant feedback; server validation is the real protection because anyone can bypass the browser."

Q. 41. How did you think about accessibility?

"Skip-to-content link, semantic landmarks, aria attributes for status and descriptions, screen-reader-only labels,." (Check each claim in the code before you say it.)

Q. 42. What do your tests prove?

"Specific behaviours: API responses and statuses with a fake model, and exact fertilizer arithmetic. They do not prove model quality, security or scale."

Q. 43. Why fake the model in tests?

"So tests are fast, deterministic and independent of TensorFlow, and they check the app's decision logic. Model quality is evaluated separately."

Q. 44. What would you monitor in production?

"Request latency and error rate by route; share of each status (diagnosed, low_confidence, no_leaf, image_quality_only); the distribution of confidence scores; and user feedback on wrong answers, to catch drift."

F. Data-engineering angle

Q. 45. How is this relevant to data engineering?

"Data quality and contracts at the boundary (upload and schema validation); strict train / validation / test separation; metrics generated by code (`evaluate_field_model.py` writes `model_card.json`, `make_report.py` writes the report) so numbers are reproducible; structured persistence with a schema and migrations; health checks; tests as guarantees. It is an application project, not a pipeline, but those habits transfer."

Q. 46. How would you make the training and evaluation a proper pipeline?

"Version the datasets and the exclusion list, parameterise the scripts (they already take paths and a seed), run them in CI or a scheduler, track runs and artefacts with a tool such as MLflow or DVC, and publish the model card automatically."

Q. 47. What data-quality problems does your data have?

"PlantDoc is web-scraped, so labels can be noisy and photos are not from my target farms. Class names had to be mapped between datasets (27 mapped). Test classes have 4–12 photos each. Eleven classes have no field data at all. I surfaced these limits in the app rather than hiding them."

Q. 48. What would you log and store if this ran in production?

"Predictions with status and confidence (already saved), model version, and, with consent, the photos for future labelling and drift checks. Today the image itself is not stored."

G. Reflection

Q. 49. What would you do differently?

"Collect more field data earlier, evaluate on field photos from the start rather than discovering the gap late, set up CI and a migration tool from day one, and add authentication before exposing history."

Q. 50. What is the biggest weakness?

Use the **honest limitations statement** in Chapter 11.

Q. 51. Why ship a model with 55.5% accuracy?

"Because it is clearly labelled a screening aid, abstains when unsure, shows measured reliability and tells people to confirm with an agriculture officer. I would rather ship a measured tool that is honest about its limits than a polished one that overclaims."

Q. 52. Show me where in the code X happens.

Be able to open, without searching: `crop_model.py → predict(), _calibrate(), explain()` · `main.py → predict_crop(), reliability_for(), fertilizer_plan()` · `fertilizer_engine.py → build_plan(), to_hectares()` · `schemas.py → PlanRequest` · `database.py → init_db(), get_session()` · `tests/test_fertilizer.py`

11 Trap questions and honest limitations

A precise limitation is stronger than an exaggerated claim. These are the questions designed to see whether you overclaim.

Trap	Honest answer
“Your resume says 75.5% field accuracy. The report says 55.5%.”	“You're right, that is a transcription error from an out-of-date README table. The measured figure is 55.5% exact class and 75.0% right crop; the generated report and model card in the repo are the source of truth, and I have corrected both.” (Fix it first so this never happens.)
“So your model is 90% accurate?”	“Only on a 380-photo clean lab sample (90.0%). On 236 held-out field photos it is 55.5%. I always name the test set.”
“Does it work for all 38 classes in real fields?”	“No. 27 classes have field photos; 11 are lab-only and the app labels them with a warning.”
“At 90% confidence it must be right?”	“No. At a 90% cut-off it answers only 13% of field photos (about 31) and 90.3% of those were right on this small set. It is not a guarantee for a new image.”
“Can a farmer follow the fertilizer plan blindly?”	“No. It converts a given dose into products. It does not choose the dose or consider soil type, season or price. The app and README say to confirm with a Soil Health Card or agriculture officer.”
“Your planner has cotton and wheat doses; your model covers tomato and apple. Why the mismatch?”	“The two modules are independent tools. The planner's references were limited to doses I could source (Maharashtra cotton and wheat). The crop model is limited by PlantVillage / PlantDoc crops. Aligning them would need new data for both.”
“Is the app connected to MySQL?”	“The code supports it through DATABASE_URL, but SQLite is the default and the tests use SQLite. I describe the database I actually ran.”
“How many users does it have?”	“None that I can claim; it is a portfolio project. I can show the tests, report and a local demo.”
“Does Grad-CAM explain the model's reasoning?”	“It shows regions that influenced a class score. It is an inspection aid, not proof the answer is right.”
“Why did lab accuracy drop after fine-tuning?”	“Training emphasised field photos, so the model moved toward the field domain: lab 93.2% → 90.0% while field 27.1% → 55.5%. It is a domain trade-off.”
“What does your non-leaf test prove?”	“Very little: 19 photos, none shown as a confident result. It is not a robust out-of-distribution test.”
“Are the reference doses verified?”	“Each has a source and a verification level. Three are verified; the wheat dose is only partly verified. Traceability to a source is not the same as agronomic validation.”
“Isn't 55.5% just bad?”	“It is modest, which is why the product is built around uncertainty: abstention at 60%, measured reliability, warnings for weak answers, and no treatment advice.”
“No authentication? Anyone can see the history?”	“Correct, the README lists this as a limitation: deploy only on a trusted network. Authentication and per-user history are the next step.”

The honest limitation statement (memorise this one)

> **SAY IT LIKE THIS**
> “The main limitation is generalisation. The crop classifier reached 55.5% exact-class accuracy on 236 held-out field photos, with a margin of about six points, and only 27 of the 38 disease classes have any field test photos. The fertilizer planner relies on a dose the farmer or a cited reference supplies; it does not decide the right dose. I designed the app to expose that uncertainty, with abstention, measured reliability, warnings and disclaimers, and I would need more field data, expert review and a security and scale review before treating it as an authority.”

If you do not know the answer

- Say what you do know, then reason: “I haven't measured that. If I had to, I would test it by …”.
- Offer to open the code or report: “The test that covers that is in test_fertilizer.py; may I show you?”
- Never invent a number. Every figure in this guide can be traced to the model card, the report or a test.

12 One-page cheat sheet

Fact	Number / answer
Model	MobileNetV2 · 224×224 input · 39 outputs (38 classes + background) · 14 crops · 9.8 MB file
Data	PlantDoc train 2,059 fit + 281 validation · PlantDoc test 236 (final only) · PlantVillage 1,520 fit, 380 held out · non-leaf 40 fit, 19 held out · 150 synthetic backgrounds
Field exact class	27.1% (64/236) → **55.5%** (131/236) · margin ≈ ±6 points
Field correct crop	47.0% (111/236) → **75.0%** (177/236)
Lab sample (380)	93.2% → 90.0%
Non-leaf (19)	0% shown as a confident result (tiny test)
Calibration	Temperature 0.95 · ECE 0.072 → **0.059**
Cut-off 60%	Answers 48.3% (114 of 236) · right 75.4% (86 of 114)
Other cut-offs	70%: 39.0% answered, 79.3% right · 80%: 26.7%, 85.7% · 90%: 13.1%, 90.3%
Field evidence	27 of 38 classes have field photos; 11 are lab-only
Weak examples	Tomato mosaic virus 0/10 · tomato healthy 1/8 · tomato bacterial spot 1/9 · best: raspberry healthy 7/7
Training	Frozen to block_12_add · top blocks trained · Adam 1e-4 · label smoothing 0.1 · class weights · early stop (best epoch 5, stopped 8)
Planner	Grades: urea 46-0-0, DAP 18-46-0, SSP 0-16-0, MOP 0-0-60, SOP 0-0-50 · 1 acre = 0.404686 ha · 40 guntha = 1 acre · splits 1–4 · dose ≤ 600 kg/ha
DAP example	100 N + 46 P_2O_5: DAP 100 kg (18 kg N) → urea 82 ÷ 0.46 = 178.3 kg
References	4 sourced doses (3 cotton, 1 wheat partly verified)
Tests	166 backend tests (37 API + 129 fertilizer; 165 pass, 1 skipped without TensorFlow) · 80-case grid = 2×2×4×5
API	8 routes (one is a liveness check) · errors 400 / 413 / 422 / 500 / 503
Database	SQLite default; MySQL via DATABASE_URL · tables crop_analyses, fertilizer_plans · image not stored

Say	Never say
“Screening aid”, “measured on 236 held-out field photos”, “55.5% exact class, 75% right crop”, “converts a dose into products”, “SQLite by default, MySQL supported”	“Diagnoses accurately”, “90% accurate” without the test set, “recommends the right fertilizer”, “production-scale”, “thousands of users”, “verified” for all references, “75.5%”, “85%”, “0.049”

Before the interview

- Resume and README corrected (Chapter 1) and pushed.
- App runs on your laptop; you know whether the model weights load and which database is configured.
- Redrawn Figures 1–3 from memory.
- Said the 30-second and 90-second pitches aloud, and the honest limitation statement.
- Opened `crop_model.py`, `fertilizer_engine.py` and `test_fertilizer.py` and explained them without notes.
- Checked the GitHub Actions tab is green.

13 Appendix A: run it and demo it in two minutes

Run locally (Windows, no Docker)

```
run_all.bat

# manual alternative
cd backend
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
uvicorn main:app --reload --port 8000

cd ..\Frontend
npm install
npm run dev
```

Open `http://localhost:8080` for the app and `http://127.0.0.1:8000/docs` for the API documentation. The README notes the Windows steps follow the existing run_all.bat and were not run on Windows while preparing that version, so test them once.

Two-minute demo script

Step	Show	Say
1	Crop Health: upload a clear tomato or apple field photo with the heat-map ticked	“Validation, model, calibration, 60% rule. It shows the measured reliability for this score and the evidence level.”
2	Upload a photo of something that is not a leaf	“It says ‘no leaf detected’ instead of guessing.”
3	Open the history list	“Every accepted upload is stored, including non-diagnoses.”
4	Fertilizer planner: choose the irrigated cotton reference, 2.5 acres, 3 splits	“Dose from a cited source; the planner converts it to kilograms per product, nets out DAP's nitrogen and builds the schedule.”
5	Enter a dose of 700	“Rejected by validation; nothing saved.”
6	Open docs/model_report.md	“Everything measured on held-out photos; I show weak classes openly.”
7	Run `python -m pytest -q`	“166 tests; the 80-case grid proves the arithmetic.”

14 Appendix B: where things are in the repository

File	Be able to say
README.md	Overview, honest limits, API table, environment variables. **Fix the 75.5% / 85.0% table and the merge-conflict leftovers.**
docs/model_report.md	Generated results: summary table, selective-prediction table, per-class table, training data counts, limitations.
backend/models/model_card.json	Temperature 0.95, threshold 0.6, evidence level per class, all measured metrics the app displays.
backend/models/field_model_metrics.json	Training log: 8 epochs, validation accuracy, temperature, test accuracy and ECE.
backend/training/train_field_model.py	Augmentation, frozen layers, class weights, label smoothing, early stopping, temperature fit, PlantDoc → label mapping.
backend/training/evaluate_field_model.py	Computes every reported metric on held-out sets and writes the model card.
backend/crop_model.py	Lazy loading, preprocessing, predict, temperature calibration, Grad-CAM, evidence levels.
backend/main.py	Routes, upload validation, status decisions, reliability and weak-answer notes, error handlers, middleware.
backend/schemas.py	Pydantic request model and cross-field rules.
backend/fertilizer_engine.py	Pure functions: to_hectares, rate_nutrient, interpret_soil, build_plan.
backend/fertilizer_data.json	Grades, units, rating limits, four reference doses with sources and verification levels.
backend/database.py	Models, engine setup, init_db with additive migration, health check, session dependency.
backend/tests/	conftest.py (fixtures, fake model), test_api.py (37), test_fertilizer.py (129).
Frontend/src/pages/	CropHealth.tsx, FertilizerAdvisor.tsx, Dashboard.tsx plus tests.
Frontend/src/lib/	api.ts (base URL, error messages), fertilizer.ts (form validation), crop.ts, a11y.ts.
.github/workflows/ci.yml	Backend pytest job and frontend lint / typecheck / test / build job.